Relatório Desafio BuzzLine

Grupo 1: Jéssica Maria e Lucas Cavalheiro

Contexto

O BuzzLine foi a última etapa do nosso processo seletivo. A tarefa era programar um robô seguidor de linha que recebemos já montado pela equipe de mecânica/eletrônica. O robô tem dois sensores de linha, dois motores, uma roda boba na frente e um botão para ligar e desligar.

O código foi escrito em C/C++ na Arduino IDE para a placa RoboCore BlackBoard Rocky, que tem um ESP32 e as pontes H dos motores na própria placa.

Como foi o processo

Começamos com um desfalque: a equipe tinha três pessoas, mas trabalhamos só em duas.
Trabalhamos na quinta-feira das 18h às 22h e na sexta-feira das 8h às 14h.
Em ambos os dias enfrentamos barreiras na estrutura do robô, de forma que grande parte do tempo disponível para calibração do código foi investida em ajustes na parte física do robô.
Os desafios encontrados foram os seguintes:

- **Troca de ambos os sensores**: o primeiro sensor foi substituído pois não lia nada além de 0; o segundo sensor foi substituído ao notar que ele não conseguia ler o range de 4095 (o máximo foi em torno de 1000).
- **Troca do botão star**t: não conseguimos comunicação com o pino do botão start, a solução foi adaptar o funcionamento para iniciar com o botão BOOT incluído na board.
- **Troca da roda boba**: a roda boba original era muito pequena para o chassi, o erro foi averiguado pela equipe de mecânica que buscou uma nova esfera e trocou suporte.
- **Mudança na estrutura do robô**: tiramos algumas partes do MDF, porque o sensor ficava mais baixo que a roda, encostava no chão e não conseguia diferenciar o claro do escuro.

Com a estrutura física corrigida, pudemos finalmente focar na lógica de calibração e controle dos motores, buscando a melhor execução possível na pista. Na sexta-feira apresentamos o robô no estado descrito na seção a seguir.

Onde paramos

Nos primeiros testes o robô tinha uma velocidade alta em um valor de KP também alto, nesse estágio, o robô consegue andar em linha reta sem muita oscilação e até completou algumas curvas. No entanto, o sucesso nas curvas era instável e não era muito reprodutível.

Para isolar o problema da passagem segura pelas curvas, nos concentramos em obter os valores mínimos de velocidade e KP, foi nessa etapa que nos dedicamos até o fim do horário disponível na sexta-feira.

A parte física está corrigida e o código está basicamente funcionando: em alguns momentos o robô segue a linha certinha, e em outros fica "sambando" (oscilando de um lado para o outro) até sair da linha. Não tivemos tempo de ajustar a velocidade e o KP com testes na pista.

O que falta para terminar

Conferir o funcionamento dos motores com base na leitura dos sensores e encontrar os valores certos para VELOCIDADE_BASE e KP na pista.

Como o robô funciona

1. Ao ligar a placa, o robô configura os pinos e começa a calibração dos sensores, que dura 8 segundos. Durante esse tempo é preciso passar os dois sensores por cima da linha e do chão, para o código aprender qual valor é a linha e qual é o chão.
2. Depois da calibração, o robô fica parado esperando o botão.
3. Ao apertar o botão, o LED da placa acende e o robô começa a seguir a linha.
4. Enquanto anda, ele lê os sensores, compara o lado esquerdo com o direito e corrige a velocidade de cada roda (controle proporcional).
5. Ao apertar o botão de novo, o LED apaga e os motores param.

Estrutura do código

O projeto tem três arquivos, que precisam ficar na mesma pasta. A pasta precisa ter o mesmo nome do arquivo .ino (buzzline).

- **buzzline.ino:** programa principal, com o setup() e o loop().
- **mapeamento.h:** pinos, ajustes (velocidade, KP, tempos) e a lista das funções.
- **controle.cpp:** o código de cada função.

Funções

iniciarPinos(): configura o botão como entrada (INPUT_PULLUP) e o LED como saída, começando apagado. Os pinos dos motores são configurados pela biblioteca da Rocky.
calibrarSensores(): durante TEMPO_CALIBRACAO lê os dois sensores sem parar e guarda o menor e o maior valor de cada um (minEsq, maxEsq, minDir, maxDir). No fim, mostra esses valores no monitor serial.
lerSensores(): lê o valor bruto de cada sensor (0 a 4095 no ESP32) e converte para uma escala de 0 a 100 usando os valores da calibração. O resultado fica em leituraEsq e leituraDir, onde 100 significa "em cima da linha".
controlarMotores(velEsq, velDir): recebe a velocidade de cada motor de -255 a 255 (positivo é para frente, negativo é ré), converte para -100 a 100, que é a escala da biblioteca da Rocky, e manda para os motores. O sinal do motor esquerdo está invertido (-velEsq) porque ele estava girando ao contrário.
controleProporcional(): calcula o erro (leituraEsq - leituraDir), multiplica pelo KP para ter a correção, tira a correção de uma roda e soma na outra. Assim o robô vira para o lado em que a linha está.
verificarBotao(): quando o botão é apertado, inverte o estado do robô (ligado/desligado), acende ou apaga o LED e espera o botão ser solto.

Como usar o código

1. Instalar a Arduino IDE.
2. Instalar o pacote de placas esp32 (Espressif Systems) no Gerenciador de Placas.
3. Instalar a biblioteca "RoboCore - Rocky" no Gerenciador de Bibliotecas.
4. Abrir o buzzline.ino com os outros dois arquivos na mesma pasta.
5. Selecionar a placa "ESP32 Dev Module" e a porta do conversor USB-UART.

6. Gravar o código. Se aparecer "Failed to connect to ESP32", segurar o botão BOOT quando aparecer "Connecting...".

Atenção: não alimentar a placa pelo conversor USB-UART e pela bateria ao mesmo tempo. Só com o USB, os motores não giram, mas dá para testar a calibração e os sensores pelo monitor serial.

Referências

- Biblioteca RoboCore_Rocky (GitHub)
- Página da BlackBoard Rocky
- Primeiros passos com a BlackBoard Rocky